



Spider
R&D Club

Spider R&D

Software Task 2

Greetings from SPIDER R&D!

Welcome to the **SPIDER R&D** Inductions '25! Over the upcoming weeks, you will take on a series of tasks designed to help you learn and explore different areas of technology and research.

The inductions will be conducted in two stages:

1. Task 1
2. Task 2

Getting Started

The tasks are designed to cater to a range of skill levels, ensuring that even beginners can learn and progress. While you may encounter challenges along the way, perseverance is key. It is normal to face difficulties; overcoming them will be part of your learning experience.

Support and Resources

If you run into any issues or need assistance, do not hesitate to reach out to any member of the club. We are here to help you navigate through the tasks and ensure a fruitful learning journey.

General Instructions:

- Carefully read and comprehend the problem statement before you begin. Misunderstandings and incorrect completion of tasks will not be acceptable. Do not hesitate to seek clarification if you're unsure about any aspect.
- The goal is for you to understand the underlying concepts and write your own code. While it's okay to look up code snippets for reference, directly copying someone else's code is forbidden.
- For complex tasks, divide them into smaller, manageable modules. Focus on completing each module individually. This approach makes the task easier and ensures steady progress and a sense of accomplishment as you complete each part.
- The Internet is a powerful tool. Use it to search for solutions to your queries and doubts. Most questions can be resolved with a quick search. However, make sure you understand the solutions you find and can explain them.
- If you encounter any issues or have any questions, feel free to contact us.

You can post your queries in the WhatsApp group or send a private message to one of us. We are here to support you and provide guidance as needed.

- Write clear and concise documentation for your code. Use comments to explain the purpose of complex code sections. This will not only help others understand your work but also aid you in recalling your thought process later.
- Test your code thoroughly to ensure it works as expected. Debug any issues that arise and verify that all parts of your code function correctly. Consider edge cases and test your code under different scenarios.
- Use version control systems like Git to keep track of changes in your code. This allows you to revert to previous versions if something goes wrong.

- The deadline for Task 2 is **12th July 2025**.

We are excited to have you on board and look forward to seeing your progress. Good luck from the Spider R&D family! 😊



Spider
R&D Club

Application Development

ClassRoom – Class Management Portal

Note: You can develop either a web application or a mobile application for completing the task.

Managing a class is harder than it should be. Announcements are scattered across WhatsApp groups, attendance is maintained separately, notes are shared through multiple platforms, and exam schedules often get lost in chat messages. As a result, both Class Representatives (CRs) and students struggle to keep track of important academic information.

Build **ClassRoom**, a centralized class management platform that enables seamless communication, attendance management, resource sharing, and exam scheduling through a single application.

Level 1

Authentication & Roles

- Implement a single Sign Up/Login system using JWT authentication.
- Support two roles:
 - **CR (Class Representative)**
 - **Student**
- CR authentication should require a predefined username and secret password.
- Students should be assigned the Student role by default during signup.
 - Use a single login route and perform role-based access control internally through the backend.

Dashboard

Announcement Bar

- CRs can create announcements containing:
 - Title
 - Description/Body
 - Timestamp
- Students can view announcements but cannot modify them.

Poll Section

- Any authenticated user can create polls.
- Users can vote only once per poll.
- Poll results should be visible after voting.
- Closed polls should be displayed in a separate read-only section.

Subjects Management

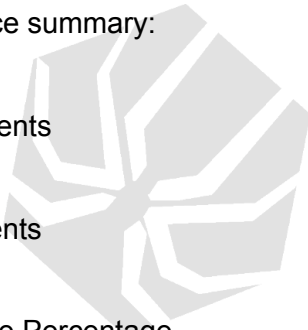
- CRs can create subjects by specifying:
 - Subject Name
 - Subject Code

Each subject should contain the following sections:

Attendance

Student Features

- View attendance history through a calendar view.
- Attendance status should include:
 - Present
 - Absent
 - No Class
- Display attendance summary:
 - Total Presents
 - Total Absents
 - Attendance Percentage



Spider
R&D Club

CR Features

- Mark attendance for all students.
- Edit previously recorded attendance.
- Attendance can be toggled for each student.
- The CR must also appear in the attendance list as a student.

Notes

- Upload and download study materials such as:

- PDFs
- PPTs
- Notes

- Display:

- File Name
- Uploader Name
- Upload Timestamp

Exams Module

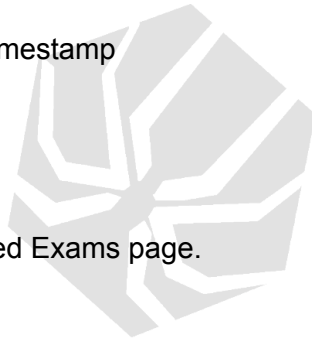
- Create a dedicated Exams page.

- CRs can:

- Add exam entries
- Delete exam entries

- Exam details should include:

- Subject
- Exam Type



Spider
R&D Club

- Date
 - Time
 - Additional Notes
- Students can view all exams sorted chronologically.
- Past exams should be visually distinguished (greyed out or shown separately).

Level 2

- **Attendance Export**

- Allow CRs to export attendance data as a CSV file containing:

- Roll Number
 - Student Name
 - Present Count
 - Absent Count
 - Attendance Percentage

Spider
R&D Club

- **Dark mode**

- Enable the user to toggle dark mode.

- **Note Notifications**

- Display a notification indicator whenever new notes/resources are uploaded.
- **Deployment Readiness**
 - Dockerize the application using Docker and Docker Compose.
- **OAuth**
 - OAuth login (Google Sign-In) as an alternative to manual signup.

Level 3

- **Real-Time Collaboration**
 - Live announcement feed using WebSockets — new announcements appear instantly without refresh.
 - Real-time poll vote counts update as users vote.
 - Typing indicators in a subject-wise discussion thread.
- **PWA**
 - Mobile PWA support with offline attendance viewing cached locally.
- **Analytics Dashboard (CR only)**
 - Attendance heatmap calendar showing class-wide presence patterns.
 - Subject-wise attendance comparison bar chart.
 - Student defaulter list — automatically flag students below a configurable threshold (e.g., below 75%).

- Exportable PDF report card per student with attendance and exam history.
- **Smart Exam Scheduler**
 - Conflict detection — warn the CR if two exams are scheduled on the same day.
 - Automated exam reminders: 24 hours before an exam, send an in-app notification to all students.
 - Students can mark exams as "prepared" / "not prepared" (private, visible only to themselves).

Guidelines

- Completing all Level 1 features will be considered completion of the task.
- Level 2 and level 3 features will provide an additional advantage during evaluation.
- Can use either PostgreSQL or MongoDB as the database (preferably PostgreSQL).
- The CR should function both as an administrator and as a student.
- Maintain a clean, intuitive, and responsive user interface.
- Follow proper folder structure and coding practices.
- You may use third-party libraries where necessary.
- If developing a web application, ensure mobile responsiveness.

Tasks will be evaluated based on:

- Completion level of the task
- UI/UX design
- Responsiveness
- Database design and API structure
- Code quality and maintainability
- Security and authentication implementation
- Creativity and additional features

Resources:

- Authentication JWT Authentication with Node.js: <https://www.digitalocean.com/community/tutorials/api-authentication-with-json-web-tokensjwt-and-passport>
- Passport.js Documentation (JWT, Session, OAuth): <https://www.passportjs.org/>
- OAuth 2.0 Fundamentals: <https://auth0.com/docs/get-started/authentication-and-authorization>
- React Official Documentation: <https://react.dev/learn>
- FreeCodeCamp React Tutorial: <https://www.freecodecamp.org/learn/front-end-development-libraries/#react>
- React Hooks Guide: <https://www.robinwieruch.de/react-hooks/>
- React Native: Style: <https://reactnative.dev/docs/style>
- Flutter: UI: <https://docs.flutter.dev/ui>
- Node.js Official Guide: <https://nodejs.org/en/docs/guides/>
- Express.js Documentation: <https://expressjs.com/>
- MongoDB with Node.js: <https://www.mongodb.com/developer/languages/javascript/node-js-mongodb-use-r-management/>
- REST API Design with Express: <https://www.freecodecamp.org/news/rest-api-design-best-practices-build-a-rest-api/>
- Database (PostgreSQL)
 - PostgreSQL Official docs: <https://www.postgresql.org/docs/>
 - PostgreSQL tutorials: <https://youtu.be/qw--VYLpxG4>

- PostgreSQL tutorial GKG: <https://www.geeksforgeeks.org/postgresql/postgresql-tutorial/>
- Database (MongoDB)
 - MongoDB Official Tutorials: <https://www.mongodb.com/docs/manual/tutorial/>
 - Mongoose ODM Guide: <https://mongoosejs.com/docs/>
 - MongoDB CRUD Operations: <https://www.geeksforgeeks.org/mongodb-crud-operations/>
- Responsive Design
 - Tailwind CSS Documentation: <https://tailwindcss.com/>
 - Responsive Web Design Basics: https://developer.mozilla.org/en-US/docs/Web/Development/Mobile/Responsive_design
- Push Notifications
- Web Push API Tutorial: <https://www.smashingmagazine.com/2017/02/push-notifications-guide/>

The Internet is your biggest learning tool. Never hesitate to search for resources of your choice.

DevOps

The Next Level



Spider
R&D Club

Yes, we are going to continue working on the Dockerization repo. You have already containerized the given full-stack application (I hope) using Docker Compose! It is now time to take it to the next level—preparing the application for a production environment.

This task will challenge your understanding of reverse proxies, Docker image optimization, horizontal scalability, and CI/CD automation—all essential for running modern cloud-native applications in production.

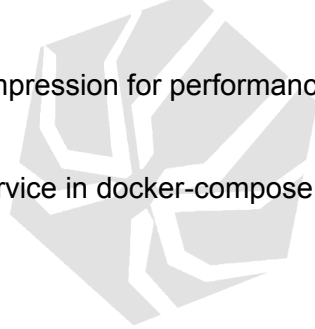
There is a new commit in the repo (<https://github.com/ilamparithi-in/cr45-reduced>) that adds logging of HTTP requests and responses to and from the server. Make sure to include that in your task 2 submission!

Task 2 Requirements

1. Reverse Proxy with Nginx and HTTPS

Set up an Nginx container that:

- Serves the React frontend at the root path (/)
- Proxies API requests (/api) to the Rust backend
- Enforces HTTPS using self-signed SSL certificates and HSTS
- Manages CORS headers properly
- Enables gzip compression for performance
- Add the nginx service in docker-compose



Spider
R&D Club

2. Jenkins CI/CD Pipeline

Automate your development workflow with Jenkins:

- Run linters and tests for both the frontend and backend on every push and pull request
- Build and push Docker images for all services to Docker Hub on every push to the main branch
- Automatically deploy updated services using Docker Compose upon successful image builds. Utilize the Docker Hub for image fetching rather than rebuilding it on the deployment.

3. Production Optimization

Review and make changes to your setup, if necessary, that optimize your application and ensures production-ready best practices, including:

- Docker Image & Container Optimization
- Nginx Performance Tuning
- CI/CD Pipeline Enhancements
- Security Best Practices in Docker, Nginx, Jenkins as well
- Observability & Resiliency in Docker and Nginx

More Brownies? Hell yeah!!

4. Horizontal Scaling

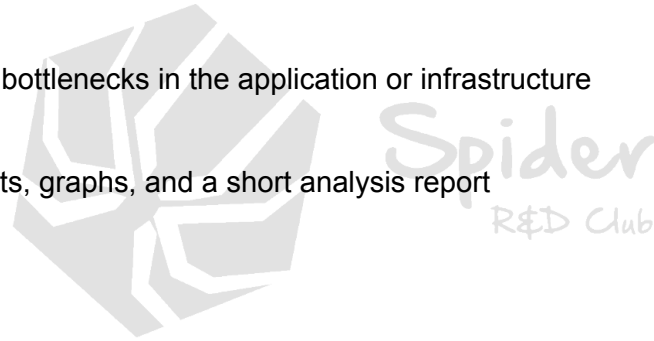
Production applications should remain available even when individual instances fail. Scale your backend service and validate that your deployment can handle failures gracefully.

- Run at least 3 backend replicas using Docker Compose
- Configure Nginx load balancing between backend instances
- Verify requests are distributed across all replicas
- Demonstrate application availability after stopping one backend instance
- Document your load-balancing and failover strategy
- Include screenshots or recordings showing traffic distribution and failure recovery

5. Performance Testing & Analysis

Evaluate the performance characteristics of your deployment and compare the impact of horizontal scaling.

- Use a load testing tool such as k6, ApacheBench (ab), JMeter, or wrk
- Benchmark both a single-backend deployment and a multi-replica deployment
- Measure requests per second (RPS), latency, and error rates
- Test with multiple concurrency levels and document the results
- Compare performance between the two deployment configurations
- Identify potential bottlenecks in the application or infrastructure
- Include test scripts, graphs, and a short analysis report



6. Security Exploration

Explore production container security practices and apply them where practical.

- Run containers as non-root users where possible
- Investigate read-only container filesystems
- Explore Linux capability restrictions
- Scan Docker images for vulnerabilities using tools such as Trivy, Gype, or Docker Scout

- Document identified risks and applied mitigations
- Include scan reports and a brief summary of findings

Submission Guidelines

- Use the same GitHub repository from Task 1 to push all updated code and configuration files.
 - Update the root level **DOCS.md** explaining the setup, architecture, and workflow with screenshots. (A screen recording would be helpful as well, upload it to Drive/YouTube)
 - It is important that whatever you do is documented in **DOCS.md** and supported by the files used (for example, the Jenkinsfile for 2, the load balancing config for 4, the load testing script for 5, and so on).
- Share the GitHub repository link as your final submission
- **1, 2, 3 must be fully implemented** for the task to be considered completion
- More attention will be given to thoughtful inclusions and modifications in 3 - 6, especially those following industry best practices. *Attention is all you need*

Resources (do not confine yourself to this list!)

1. Nginx

<https://nginx.org/en/docs/>

<https://www.youtube.com/watch?v=ilnUBOVeBCc>

<https://medium.com/@mathur.danduprolu/securing-your-web-server-with-nginx-https-and-best-practices-part-5-7-99ad19bf5b1f>

2. Jenkins

<https://www.youtube.com/watch?v=6YZvp2GwT0A>

<https://medium.com/@prateek.malhotra004/essential-jenkins-best-practices-for-developers-streamline-your-ci-cd-process-dc3aa38f9928>

<https://www.jenkins.io/doc/book/pipeline/pipeline-best-practices/>

3. Docker

<https://snyk.io/blog/10-docker-image-security-best-practices/>

<https://sysdig.com/learn-cloud-native/dockerfile-best-practices/>

<https://docs.docker.com/docker-hub/>



Ingredients for Biryani

Load balancing: https://nginx.org/en/docs/http/load_balancing.html

Load testing: <https://grafana.com/docs/k6/latest/>

Security: <https://trivy.dev/docs/>

You are free to use any alternatives to the software listed here.

Blockchain Development

Full Time

On-Chain Prediction Markets for the World Cup

The whistle blows. Ninety minutes of football, and somewhere a contract is waiting to pay. No bookmaker, no house edge, no settlement desk taking three days to clear your winnings. Just stablecoin locked in escrow, odds set by the crowd, and a result written to the chain the moment the final whistle sounds. The world is watching the World Cup. Your job is to let the world bet on it, trustlessly. Build the market, price the odds, and teach a machine to call the result.

You will build a prediction market where users stake a stablecoin on the outcome of real World Cup matches. Markets are collateralised in USDC, priced by a market mechanism, and resolved by an oracle. The build escalates across three levels, from a simple pooled market with a trusted referee to a live-odds market settled by an optimistic oracle that an autonomous AI agent drives.

The timing is the point. The group stage runs to 27 June and the knockouts run through to the final on 19 July, so your markets can open before kickoff and pay out against results that actually happened. A market that settles because Brazil really did draw Morocco is a far better demo than a toy that never resolves. Everything lives on Base Sepolia with real testnet USDC. Simulated payouts will not be accepted.

Level 1: The Betting Pool

Parimutuel pricing and a trusted referee. Get the core market loop correct, end to end.

Core contract: PredictionMarket.sol. A market for a single fixture that must:

Support three outcomes per match (Home win, Draw, Away win). Football draws are real; a binary Yes/No market is wrong here unless your question is strictly binary.

Accept USDC stakes on one outcome until a closeTime (kickoff). No bets after close.

Price parimutuel: each outcome holds a pool. After resolution, the total staked, minus a configurable protocol fee, is split among the winners in proportion to their stake.

Resolve via a trusted resolver address that sets the winning outcome after full time.

Pay out through claim(), with a reentrancy guard and the .call() pattern for transfers.

Handle the edge cases: nobody staked the winning outcome (refund or roll over), match cancelled or postponed (full refund), claiming twice (revert), staking after close (revert).

Factory: MarketFactory.sol. Anyone can deploy a market per fixture; track and return markets by status. Emit events on every state change: MarketCreated, BetPlaced, MarketResolved, Claimed, MarketCancelled. Collateral is USDC (or a mock USDC ERC-20) on Base Sepolia.

Tests (Foundry): stake, resolve, claim happy path; a draw resolves and pays correctly; refund when no one backed the winner; double-claim reverts; bet-after-close reverts. Use vm.warp() to move past closeTime and resolution.

Frontend: connect wallet, list open markets, approve and stake USDC, show each outcome's pool and implied odds, show resolution and claim.

Level 2: Live Odds and Trustless Settlement

A market maker for moving odds, and an oracle nobody has to trust.

Pricing: add a CPMM. Introduce a constant-product market maker for outcome shares so odds move live as people trade. Buying a “Home win” share pushes that price up; prices across outcomes stay between 0 and 1 and sum to roughly 1. A creator seeds initial liquidity in USDC. Document how liquidity provision works and the tradeoffs you accept (full LP withdrawal is not required, but reason about it in your writeup).

Resolution: an optimistic oracle. Replace the trusted referee. Anyone may propose the outcome by posting a bond. A dispute window opens. If undisputed, it finalises. If disputed, escalate (to a fallback resolver or a simple vote) and slash the loser’s bond. The market reads the finalised outcome from the oracle, never from an owner.

Tests: CPMM price invariants; the full propose, dispute, finalise flow using `vm.warp()` through the dispute window; bond slashing on a wrong proposal.

Frontend: a live odds view, buy and sell shares, an oracle status panel (Proposed / In Dispute / Finalised) with a countdown to the dispute deadline.

Level 3: The Autonomous Referee

Take the human out of the loop. An agent reads the result and settles the market.

Build an off-chain resolver agent (TypeScript). When a market’s close time passes, the agent fetches the real match result from a football data API, decides the outcome, and submits it on-chain as the proposer into your optimistic oracle, posting the bond. The happy path has no human in it.

Run it autonomously: poll for markets needing resolution, resolve each, and log every action with its transaction hash.

Budget guard: before proposing, the agent checks its own balance and exits cleanly with a clear message if it cannot cover the bond and gas. The resolver holds its own wallet, separate from any user wallet.

Live dashboard (React): markets with live odds, a detail view with an odds chart, and an agent activity feed that shows the resolver fetching results and proposing outcomes in real time (SSE or polling), with links to each transaction on the Base Sepolia explorer.

Bonus: a simple trading bot that takes positions from a heuristic or an external odds feed, so the market shows two-sided flow.

Design notes you will be judged on

Three outcomes, not two. Handle Home / Draw / Away, or specify strictly binary questions with care.

Money edges: protocol-fee accounting, no-winner refunds, cancelled or postponed fixtures, double-claim protection, late bets.

Front-running the result: consider whether anyone can act on a known result before it finalises, and how your oracle window defends against it.

Secrets: no hardcoded private keys, `.env` never committed, the resolver agent holds its own key.

Note

A fully working **Level 1**, deployed and verified on Base Sepolia, counts as completing the task.

Level 2 gives you a clear edge. **Level 3** (the autonomous resolver and dashboard) is where the strongest submissions separate. Real testnet, real USDC, payouts visible on-chain. Simulating transfers will not be accepted.

Task will be evaluated based on:

Area	What we check
Smart contract	Market state machine, three-outcome handling, parimutuel and CPMM math correctness, reentrancy guard and .call() on payouts, access control, fee and refund edge cases.
Oracle	Optimistic propose, dispute and finalise logic, bond posting and slashing, market consumes only the finalised outcome.
Tests	Foundry unit tests using vm.warp() across close and dispute windows; forge coverage at least 80% on the core contract.
Deployment	Contracts live and verified on Base Sepolia, a clean deploy script, addresses recorded in the README.
AI resolver (Level 3)	Autonomous result fetch and on-chain proposal, an action log with transaction hashes, a working balance guard.
Frontend	Wallet connects, USDC approve and stake works, odds display correctly, resolution and claim are visible to the user.
Code quality	Clean README with addresses, events on every state change, no hardcoded keys, .env not committed, a configurable constructor.

Submission

Repository named [Name]_Spider_Task_2. Suggested layout: /contracts, /test, /script, /frontend, /agent.

README with deployed contract addresses, setup and run instructions, and a demo: a deployed link or a 2-minute screen recording.

Writeup named [Name]_[RollNo]_PredictionMarket.md covering your two big design decisions (pricing model and oracle model), the edge cases you handled, and what you would build next.

AI tools are permitted. Understand every line; we will ask about it in the interview.

Resources

Foundry docs: <https://getfoundry.sh/>

Circle testnet USDC faucet (select Base Sepolia): <https://faucet.circle.com/>

Base Sepolia explorer: <https://sepolia.basescan.org/>

viem docs: <https://viem.sh/>

ethers v6 docs: <https://docs.ethers.org/v6/>

OpenZeppelin ReentrancyGuard and ERC20:

<https://docs.openzeppelin.com/contracts/5.x/>

AMM maths (Uniswap v2 core) and LMSR: background for market pricing.

Optimistic oracle pattern (UMA): background for trustless resolution.

Football data API (for example [football-data.org](https://api.football-data.org/)): for the resolver agent.

Polymarket and Augur: conceptual references for prediction markets.

CyberSecurity:

Octo-shell: a UNIX based custom bare bones shell

As you and your fellow SPIDER developer were sitting and discussing why anyone would ever willingly learn Javascript and joking that you would rather fight terrorists than learning it lol, there was a huge BOOM!

The campus has been breached and **real terrorists have actually entered the campus!** But these weren't your normal terrorists. They knew the sheer intellectual caliber of NIT Trichy students. Fearing that you might communicate with each other or the outside world to coordinate a counter-attack, they decided to take absolute control of the main server rooms.

Being dangerously smart, they established a digital net over the campus Local Area Network (LAN). Using Wireshark and Tshark scripts running in promiscuous mode across the campus Wi-Fi access points, **they are actively sniffing every single packet. If any student transmits unencrypted data or files, the adversaries instantly intercept it,** pinpoint their exact coordinates, and off them. Basically, if you send so much as a "hello" over plaintext, they'll find you and probably force you to use windows-11 for the rest of your life.

You and your friend managed to grab your Linux laptops and slip out of the lab just in time. To avoid mass detection, the two of you have split up, hiding in different hostel blocks across the campus. **You have no cellular data, and you have no internet. You only have the local Wi-Fi router humming in the corridor, completely monitored by the enemy.**

As an elite operative of the SPIDER group, **your task is to develop a custom, operational shell from scratch** so you can re-establish contact and coordinate an escape plan. The fate of the college **depends on you.**

Level 1: The Core shell

Standard shells leave history files and logs that the adversaries can trace. Given the ultimate developer you are, you've decided the only safe way forward is to **develop your own custom, bare-bones shell from scratch.**

- implement a standard **command line REPL** (Read-Eval-Print Loop) in pure C
- **System Commands:** For normal Linux tools (like `ls`, `pwd`, or `cat`), your shell needs to use the `fork()` system call to spawn a child process, and `execvp()` to run the actual binary.
- **Built-in Commands:** Not everything can be an external process. You must implement `cd` directly inside the shell using the `chdir()` system call. When you type `exit`, just make sure your C code frees any allocated memory before shutting down cleanly.

NOTE: for this phase, **directly follow and finish the “build your own shell” codecrafters challenge**, this is very much necessary if you want to proceed for the next level

Level 2: The radio...

You have your bare-bones shell up and running. But a shell alone won't help you coordinate an escape plan across the hostel blocks. You need a way to send files, text messages regarding the information about the terrorists.

Your Objective: Develop an ephemeral, point-to-point file transmission mechanism natively compiled within your `octo-shell`. To keep the application footprint small and avoid complex concurrency stuff, **communication will run over a dual-terminal system.**

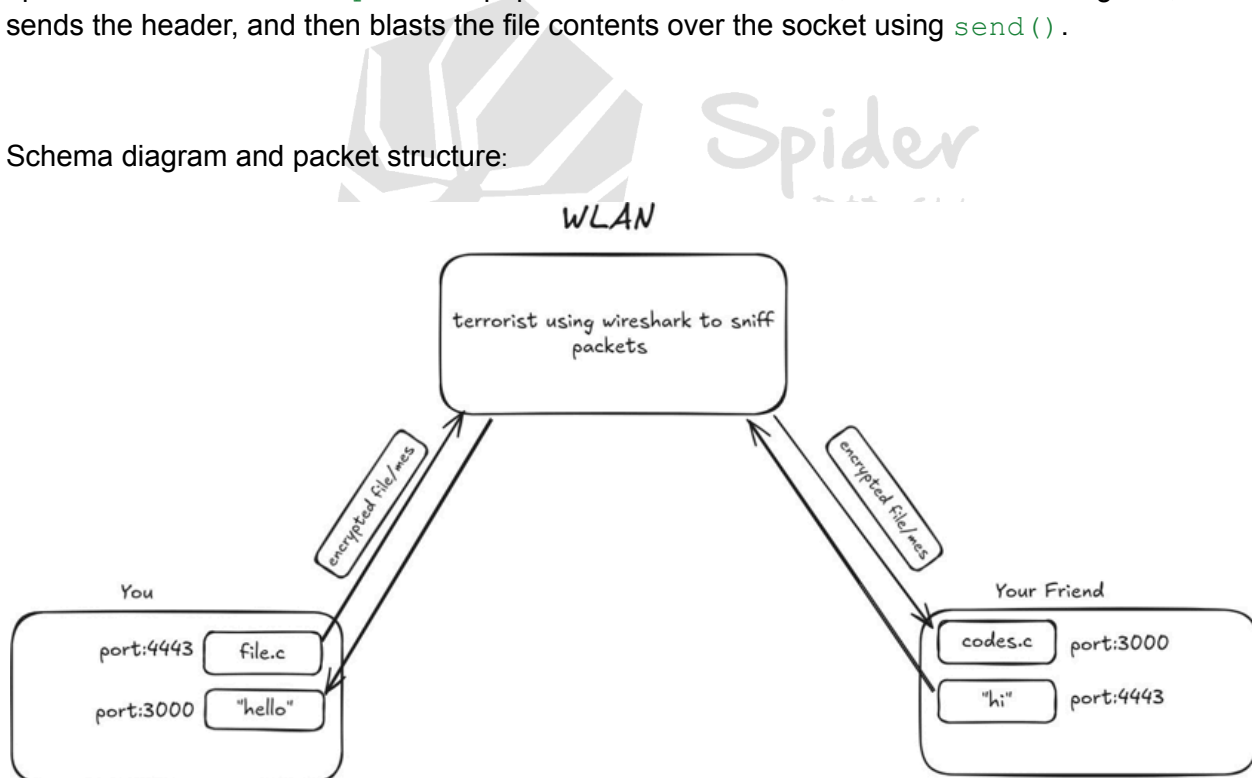
The Wire Protocol (Packet Structure): TCP is a continuous stream of bytes. If you just blast raw file data over the network, the receiver has no idea what the file is named or when the transmission ends. You must define a strict metadata header (a C `struct`) to send right before the file. **Watch out for compiler padding; your struct must be exactly 72 bytes!**(this is really to important, u will understand once you start building it)

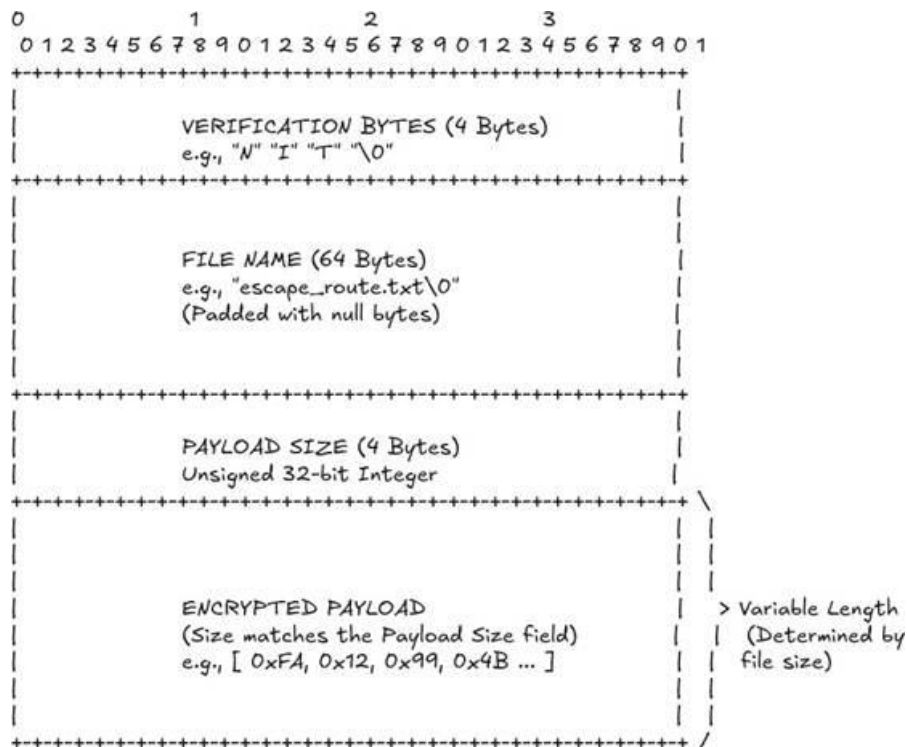
- **verification Bytes (4 Bytes):** e.g., `"NIT\x00"`. This validates that the connection is actually coming from a friendly SPIDER node, not a terrorist port scanner.
- **File Name (64 Bytes):** The target name of the file so the receiver knows what to save it as.
- **Payload Size (4 Bytes):** A `uint32_t` number telling the receiver exactly how many bytes to read before closing the connection.

The Listener (Terminal A): Your teammate opens a separate terminal window running `octo-shell` and executes `nittalk -listen 4443`. This command must block (freeze) on the `accept()` system call, waiting silently for an inbound connection. When a connection arrives, it must read the 72-byte header, verify the bytes, open a new file using `fopen()`, and use `fwrite()` to commit the incoming stream to the physical hard drive.

The Sender (Terminal B): Once you find your teammate's IP address, you blast the file directly across the LAN using `nittalk -s <Target_IP> -f payload.c`. Your shell opens the local file with `fopen()`. It populates the header struct, connects to the target IP, sends the header, and then blasts the file contents over the socket using `send()`.

Schema diagram and packet structure:





Very Important point : Different CPU architectures (like an ARM Mac vs an x86 Linux machine) store multi-byte integers backwards from each other (Little-Endian vs. Big-Endian). If you transmit this `uint32_t` raw, a 1,024-byte file might be read as a 17-billion-byte file on the other side, causing an immediate buffer overflow. You must convert this size integer to Network Byte Order using `htonl()` before sending, and convert it back to Host Byte Order using `ntohl()` upon receiving.

If you dont understand what i just said watch this video:
<https://youtu.be/NcaiHcBvDR4?si=Qxkb672CQH8tSn8L>

Level 3: Encryption..

Alright, your `nittalk` radio is live. You can blast files across the hostel blocks. But there is a massive, glaring problem: the data is flying over the LAN in pure plaintext. If you send `escape_plan.txt`, the insurgents monitoring the Wi-Fi router aren't just going to see *that* you sent a file, they are going to read every single word of it, find you and slime yo ass with no mercy.

JavaScript developers would panic here. They'd cry about not having internet access to run `sudo apt install libssl-dev` to use pre-built encryption libraries. But you and your

friend aren't normal developers. You are the ultimate low-level systems hackers. Relying on someone else's bloated cryptographic library feels like a personal insult to your loyalty for Fabrice Bellard . You've decided to build your own encryption scheme entirely from scratch in pure C.

Step 1: The Key Exchange Protocol

To encrypt data, both terminals need an identical secret key, but you cannot transmit this key over a compromised network. You will implement a cryptographic handshake based on modular arithmetic:

1. **Global Public Parameters:** Both terminals **hardcode two public numbers**: a large prime number p and a generator base g .
2. **Private Key Generation:** * Your terminal generates a secret random integer a .
 - Your friend's terminal generates a secret random integer b .
3. **Public Key Calculation & Transmission:**
 - Your terminal computes $A = g^a \pmod{p}$ and sends A to your friend.
 - Your friend's terminal computes $B = g^b \pmod{p}$ and sends B to you.

WARNING ON C MATH: Do **NOT** attempt to use the standard `pow()` function from `<math.h>` for this. Cryptographic math involves massive numbers. If you attempt to calculate g^a normally, the intermediate value will instantly overflow a 64-bit integer container (`uint64_t`), returning completely corrupted garbage data before the modulo is even applied. You must research and implement a **Modular Exponentiation (Square-and-Multiply)** algorithm from scratch to keep the calculations safely inside your RAM bounds.

4. Shared Secret Derivation:

- You compute the shared key: $K = B^a \pmod{p}$.
- Your friend computes the shared key: $K = A^b \pmod{p}$.

Mathematically, both calculations yield the exact same value “K”. An adversary sniffing the network intercepts only g , p , A , and B . Without knowing the private exponents a or b , computing K requires solving the discrete logarithm problem, which is computationally infeasible.

Step 2: The Keystream Generator

Once both sides share the integer K, it must be transformed into a continuous stream of pseudorandom bytes (a keystream) matching the size of the file being sent.

- Use K to seed a custom deterministic random byte generator (such as a Linear Congruential Generator or a basic shift-register algorithm implemented natively in C).
- For every byte of the file processed, the generator must output exactly one byte of pseudorandom data.

Step 3: The XOR Stream Cipher

With the keystream ready, the data masking happens at the bit level before transmission:

- **Encryption:** Take a raw byte from the source file, apply the C bitwise XOR operator (^) against the corresponding keystream byte, and transmit the result over the socket:
$$\{\text{Cipher_Byte}\} = \{\text{Raw_Byte}\} \text{ XOR } \{\text{Keystream_Byte}\}$$
- **Decryption:** On the receiving side, your friend regenerates the identical keystream using the same shared secret K. They apply the XOR operator to the incoming network bytes to recover the original file:
$$\{\text{Raw_Byte}\} = \{\text{Cipher_Byte}\} \text{ XOR } \{\text{Keystream_Byte}\}$$

To the intercepting adversaries, the network traffic now appears as entirely random binary noise, rendering their sniffers completely useless.

Note: Level 3 is really easy to implement if you have done cybersecurity basic task

Level 4: RUN! Before they get you...

Your encrypted `nittalk` radio is up, and files are moving safely in the dark. But the insurgents aren't stupid. They know someone is operating on the network. They are actively running `nmap` ping sweeps and SYN scans across the campus subnet, trying to find open ports and active laptops.

If they find your IP address, it's only a matter of time before they close in. You need an early warning system, once you know they're onto you, you gotta take off and hide somewhere else!

Your Objective: Develop `tripwire`, a standalone, passive Intrusion Detection System (IDS) that monitors raw network traffic and alerts you the second an unauthorized machine probes your laptop

Raw Sockets: Standard sockets (`SOCK_STREAM`) are useless here; they only wake up *after* a full connection is made. You must use a Raw Socket (`SOCK_RAW`) to intercept packets directly from the Network Interface Card, *before* the operating system processes them. Because this requires kernel-level access, `tripwire` must be executed with root privileges (`sudo`).

Memory Parsing: When a packet hits the raw socket, it won't be nicely formatted text. It will be a raw array of bytes (`unsigned char buffer[]`). You must use pointer arithmetic and cast this memory buffer into standard Linux network structures (like `struct iphdr` from `<netinet/ip.h>`) to make sense of the data.

The Interrogation: Extract the Source IP address (`saddr`) from the IP header of every incoming packet and convert it to a readable string using `inet_ntoa()`.

The Tripwire: Hardcode your known teammate's IP address into the `tripwire` code. Compare every incoming Source IP against your teammate's IP.

The Alarm: If an incoming packet's Source IP does *not* match your teammate's IP, it means the enemy is probing your machine. `tripwire` must instantly print a high-priority, color-coded alert to the terminal.

NOTE: here the terrorist are only doing an passive scan to check all the active IP address,

Scenarios to visualize:

Case 1: Your Friend Sends an Encrypted File

Your friend across the courtyard wants to send you the `escape_route.txt` file. They run their `nittalk -s <Your_IP> -f escape_route.txt` command.

1. Your friend's laptop shoots a TCP SYN packet across the Wi-Fi. It hits your laptop's antenna.
2. The Linux Kernel receives the packet and copies it into two paths simultaneously.

3. The Security Camera (`tripwire`):

- `tripwire` catches the raw packet.
- It strips the IP header and looks at the Source IP. It sees your friend's IP address. It checks its whitelist. It matches! phew!
- `tripwire` says, "This is friendly," and stays completely silent. No alarms are triggered.

4. Inside the shell running `nittalk`:

- A microsecond later, the Linux Kernel hands the connection to Port 4443.
- `nittalk` wakes up from its `accept()` freeze.
- Your friend's shell sends the 72-byte Phase 2 Header. `nittalk` checks the Magic Bytes (`NIT\x00`). They match perfectly.
- `Octo-shell` accepts the Phase 3 encrypted payload, runs the XOR math to decrypt it, and saves `escape_route.txt` to your hard drive.

The Result: A clean, silent, secure file transfer.

Case 2: The Terrorist Runs a Port Scan

The terrorists are sitting in the server room (IP: `192.168.1.99`). They run a massive `nmap` port scan across the entire campus Wi-Fi, looking for anyone hiding.

1. **The Approach:** The terrorist's automated script shoots a TCP SYN packet at your laptop, probing to see if Port 4443 is open.
2. **The Fork:** The Linux Kernel receives the packet and copies it into two paths simultaneously.
3. **`tripwire`:**
 - `Tripwire` catches the raw packet.
 - It strips the IP header and looks at the Source IP. It sees `192.168.1.99`.
 - It checks its whitelist. **NO MATCH.**
 - **TRIPWIRE TRIGGERED!** `tripwire` instantly blasts a massive red alert to Terminal 2: `[WARNING] Hostile reconnaissance detected.`
 - Because we removed the Kill Switch, Port 4443 is still open. The Linux Kernel hands the connection to `octo-shell`.
 - `octo-shell` wakes up from its `accept()` freeze and expects the 72-byte Phase 2 Header.

- The terrorist's `nmap` script doesn't know about Project octo-shell; it just sends standard garbage data (or nothing at all) to see if the port is alive.
- `nittalk` looks for the Magic Bytes (`NIT\x00`). It sees garbage.
- `nittalk` says, "You aren't SPIDER," and immediately executes `close(client_socket)`, dropping the connection to protect the shell from buffer overflows.

The Result: The terrorist's scan gets dropped by `octo-shell`. Time to move to some other place!!

NOTE: remember that `nittalk` is an builtin for the octo-shell...not an standalone application. whereas `tripwire` is...

Task will be evaluated based on:

Area	What we check
Code readability	Proper memory management (freeing pointers on exit to prevent leaks), correct use of compiler directives for the 72-byte packed struct, and clean implementation of system calls without zombie processes.
Genuine effort	Handling file chunking for large payloads (preventing stack overflows), successfully parsing raw memory buffers for the IDS tripwire, and ensuring the custom XOR crypto perfectly restores binary files without null-byte truncation
Modularity	Code is cleanly separated into distinct source and header files (e.g., <code>crypto.c</code> , <code>shell.c</code> , <code>network.c</code>) rather than one massive file, paired with a functional <code>Makefile</code> for seamless compilation..

Submission format:

- **Repository Layout** : Name the repository `[Name]_Spider_Task_2`. The project should follow a standard C directory structure: `/src` for source files, `/include` for headers, and `/tests` for any dummy files used during testing.

- **README & Demo:** Include a `README.md` with explicit compilation instructions (e.g., `make clean && make`). Provide a link to a 2-minute screen recording demonstrating the dual-terminal file transfer, the successful decryption of the file, and the `tripwire` alert flashing red when an external ping hits.
- **Technical Writeup :** Submit a document named `[Name]_[RollNo]_OctoShell.md`. This writeup must cover the mathematical approach used for the key exchange, the specific edge cases handled (like buffer constraints or struct padding), and the exact system calls used to manipulate the raw sockets.
- **The Interrogation Clause:** AI tools are permitted for debugging, **but vibecoding is strictly forbidden. You must understand every single pointer, bitwise operator, and memory allocation in your codebase. We will ask you to explain your packet structure and the schema during the interview.**

Resources(follow in order)

1) These are some really good lectures from low level academy, install VLC media player and directly watch them, directly do “open folder” in VLC for continuous streaming, WE STRONGLY RECOMMEND to watch all the lectures and get your C programming skills strong, In the cybersecurity club most of the time we bang our heads on the keyboard trying to understand what kind of crappy C code did open source organizations write to exploit vulnerabilities in their software.

https://drive.google.com/drive/folders/1oZ10WGSf6zV-Fwlw-XpRax9l3yn3hg_b?usp=drive_link

2) <https://youtu.be/LDhoD4lVEIk?si=xC9CJMRqSbAbPTEb> :Watch this videos and other videos from this channel , some really cool stuff

3) <https://beej.us/guide/bgnet/html/split/> : one the best resources for network programming, if you don't understand certain stuff from the lectures provided then you refer to this to gain better understanding.

4) <https://app.codecrafters.io/courses/shell/overview>: Complete this codecrafters challenge, this is an absolute must for the reviewing of your task. You must implement the builtin by modifying the shell after you complete the challenge.

5) https://youtu.be/rJrd2QMVbGM?si=30v93a7N_bD28DHG: this is also an an must, PI will be mainly to test your programming concepts and how fast can u

understand codebases, that comes from having a very strong fundamentals of operating system concepts and memory management

UI UX

Overview

In this task you will be designing a Mobile expense tracker application. The point of Task 2 is to push you somewhere new. If you're comfortable the whole time, you're not learning. We'd rather see a flawed-but-bold experience than a safe-but-polished one

Mobile Expense Tracker & Budget Manager

You are designing a mobile-first personal finance application. The interface must be optimized for a standard mobile viewport (dimensions: 414 x 896 px, as specified for mobile environments).

Required Screens & Core Views

1. The Financial Dashboard (Home): The user's main financial command center. It must display their total current balance, an expandable quick-glance visual breakdown of monthly spending categories (e.g., Food, Rent, Entertainment), and a "Recent Transactions" ledger.
2. The Budget Planner View: A dedicated screen where users can set, monitor, and adjust monthly budget thresholds. This must include progress indicators or visual gauges showing how close they are to exceeding their set limits (e.g., "You have spent 82% of your dining budget").
3. The Transaction Logger: An interactive modal for adding a new expense. It must include fields for Amount entry, Category selection (using clear tags or icons), Date/Time, and optional receipt image upload attachment.
4. Analytics & Spending Insights: A view dedicated to financial trends. It should display a simple graph tracking monthly savings over time, alongside a breakdown of their "Top Spending Sinks" and recurring subscription fees.
5. Feel free to design additional pages. There's always room for creativity.

Technical & System Requirements

- Auto Layout Integration: Every frame, input field, and list item must be entirely built using nested Auto Layout to ensure dynamic content adaptation.

- Component Tokens & States: Expand your Figma component library to include financial asset elements, input field states and interactive toggle variations.

Non-negotiables

1. Pre-Design Strategy (Required)

Before executing high-fidelity interfaces,

- Detailed User Flow Diagram: Map out the exact step-by-step logic of a user inputting a transaction, showcasing handling for edge cases such as missing input data or an over-budget trigger warning.
- Lo-Fi Wireframes: Submit clean, monochrome structural blueprints of all four screens within a separate, dedicated frame in your Figma file before styling.

2. Interactive Prototyping:

Smart Animate Transitions: Utilize smooth Smart Animate interactions to move between screens (e.g., dynamically expanding a transaction line-item into its full detail view).

Micro-interactions: Implement at least 3 distinct micro-interactions beyond screen navigation, such as an animated button loading state, or some smooth sliding animations.

Pre-design work (Required Deliverables)

Prioritize strategic planning before commencing screen design:

Moodboard (1 page): Include references, color directions, type inspiration, and motion references. Source material from Dribbble, Mobbin, Pinterest, and authentic Wrapped screenshots from previous years.

User Journey (1 page): Define the emotional arc, including the build-up and payoff. Map this progression as a simple flow diagram with corresponding notes on the intended user experience and emotional response for each screen.

Lo-fi Wireframes: Submit rapid, black-and-white iterations of all screens prior to high-fidelity design. These must be presented in a separate frame.

Restrictions

- The use of AI-generated user interfaces is strictly prohibited.
- Copying existing Wrapped screens is prohibited: Draw inspiration, but refrain from direct tracing or duplication.
- All illustrations and graphics must be either custom-made or sourced exclusively from credited free repositories (e.g., unDraw, Blush, Storyset).

BONUS TASK (Optional)

Frontend Implementation

Earn brownie points. Pick any 2 screens from your recap flow and build them as a real working web page.

- Use HTML/CSS/JS or React (your choice)
- Animations should match your Figma prototype as closely as possible
- Use Framer Motion (React) or GSAP (vanilla JS) for animation
- Mobile-first should look right on a phone-sized viewport

Resources *(Watch what you need, when you need it don't try to watch everything before starting.)*

1. Figma Fundamentals *(If you've never opened Figma, start here.)*

- **Kevin Stratvert**: Figma Tutorial for Beginners [\[watch\]](#)
- **DesignWithArash**: Figma Crash Course for Beginners [\[watch\]](#)

2. Auto Layout

- **Figma**: A Guide to Auto Layout [\[watch\]](#)
- **AlienaDesign**: Figma Auto Layout Tutorial [\[watch\]](#)

3. Components & Variants *(How to build buttons, cards, and stat blocks)*

- **DesignWithArash**: Master Figma Components [\[watch\]](#)
- **Soren Iverson**: Build a Figma Card Component [\[watch\]](#)

4. Prototyping & Smart Animate

- **Tim Gabe**: Crash Course: Animations in Figma [\[watch\]](#)
- **Saptarshi Prakash**: Figma Animations for Beginners [\[watch\]](#)

5. Moodboards & Design Process

- **Mizko**: How To Create a Mood Board (Design Process for App UI Projects) [\[watch\]](#)
- **DesignWithCanva**: How to Create a Moodboard in Figma (Beginner to Advanced) [\[watch\]](#)

6. Framer Motion (for the bonus task, React)

- **Net Ninja**: Framer Motion for React (full 18-part series) [\[watch\]](#)
- **PedroTech**: React Animation Course with Motion [\[watch\]](#)
- **Manu Arora**: Framer Motion Basics [\[watch\]](#)

7. GSAP (for the bonus task, vanilla JS / HTML)

- **Hyperplexed**: Master GSAP Animations (Basics, Complete Course Part 1) [\[watch\]](#)
- **JavaScript Mastery**: Web Animations Full Course [\[watch\]](#)

Submission

- Figmafile link: Ensure view access is enabled and verify the link in an incognito tab before submission.
- Prototype link: The playable, swipeable version of your design.
- Short writeup (maximum 1 page): A concise document covering design rationale, key decisions, and proposed future improvements.
- Bonus code: Provide the GitHub repository link.

— **Spider R&D** —